

Beyond GLMs with splines

Bert van der Veen

University of Bayreuth

Resources

- ▶ **Fieberg, J.** (2024). *Statistics for Ecologists*.
<https://statistics4ecologists-v3.netlify.app>
 Ch. 4: Non-linear models, intuitive treatment of splines and basis functions
- ▶ **Simpson, G.** Physalia GAM course.
<https://github.com/gavinsimpson/physalia-gam-course>
 Comprehensive practical course with `mgcv`; slides and code freely available
- ▶ **Wood, S.N.** (2017). *Generalized Additive Models: An Introduction with R*, 2nd ed. Chapman & Hall.
 The definitive reference; covers all theory and `mgcv` in depth

R packages for GAMs

Fitting

- ▶ `mgcv`: the standard; most features, best documented
- ▶ `gamm4`: GAMMs via `lme4` backend
- ▶ `glmmTMB`: GAM smooths + arbitrary GLMM structure

Visualisation & diagnostics

- ▶ `gratia`: `ggplot2`-based smooth plots, `draw()`, `appraise()`
- ▶ `DHARMa`: simulation residuals
- ▶ `ggeffects`: marginal predictions for `mgcv` models

We will use `mgcv` for fitting and `gratia` for visualisation

Where we are

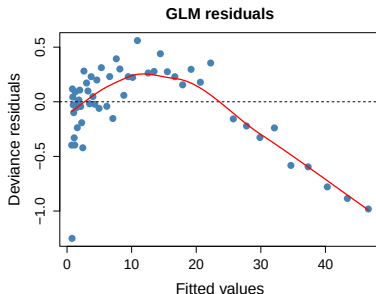
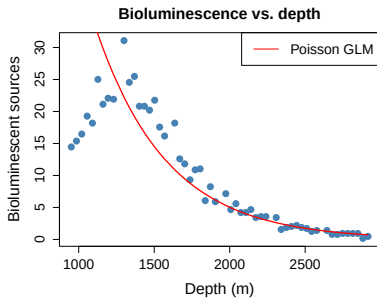
Model	Non-linear covariates	Correlation
GLM	No	No
GLMM	No	Yes (random effects)
GAM	Yes (smooth functions)	Optional

Where we are

Model	Non-linear covariates	Correlation
GLM	No	No
GLMM	No	Yes (random effects)
GAM	Yes (smooth functions)	Optional

- ▶ GLMs and GLMMs have non-linearity through the link function, but assume covariates enter the linear predictor **linearly**
- ▶ Many ecological relationships are non-linear

Non-linearity in ecology



A Poisson GLM misses the non-linear relationship.

Penalized regression

Linear regression minimizes:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 \quad (1)$$

Penalized regression

Linear regression minimizes:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 \quad (1)$$

Ridge regression penalizes the coefficients:

$$\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda \|\boldsymbol{\beta}\|^2 \quad (2)$$

- ▶ $\lambda \geq 0$ controls the strength of penalisation too zero
- ▶ $\lambda = 0$: ordinary model

Ridge regression: solution

The ridge estimator has a closed form:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^{\top} \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{y} \quad (3)$$

Ridge regression: solution

The ridge estimator has a closed form:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (3)$$

- ▶ The $\lambda \mathbf{I}$ term regularizes the problem; helps when $\mathbf{X}^T \mathbf{X}$ is ill-conditioned (e.g., with multicollinearity)
- ▶ Very useful with many covariates ($p > n$)

Ridge regression and GLMMs

Ridge regression is similar to a mixed model that assumes:

$$\beta \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda} \mathbf{I}\right) \quad (4)$$

Ridge regression and GLMMs

Ridge regression is similar to a mixed model that assumes:

$$\beta \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda} \mathbf{I}\right) \quad (4)$$

For a Gaussian model with **known** λ : minimizing the penalized sum of squares gives the posterior mode (MAP estimate) of β , i.e. the conditional mode given the data.

Ridge regression and GLMMs

Ridge regression is similar to a mixed model that assumes:

$$\beta \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda} \mathbf{I}\right) \quad (4)$$

For a Gaussian model with **known** λ : minimizing the penalized sum of squares gives the posterior mode (MAP estimate) of β , i.e. the conditional mode given the data.

The penalty λ controls variance: small $\lambda \Leftrightarrow$ large variance $\Leftrightarrow$ flexible

Ridge regression in mgcv

```
library(mgcv)
```

```
# Scale predictors so penalty is on a comparable scale
```

```
X <- scale(as.matrix(data[, c("x1", "x2", "x3")]))
```

```
data$X <- X
```

```
# paraPen adds lambda * beta' I beta to the log-likelihood
```

```
model <- gam(y ~ X, family = poisson, data = data,  
             paraPen = list(X = list(diag(3))),  
             method = "REML")
```

The distinction does not quite hold for GLMMs

- ▶ **Ridge:** λ is fixed externally (cross-validation, AIC); β is optimized directly with no marginal likelihood

The distinction does not quite hold for GLMMs

- ▶ **Ridge**: λ is fixed externally (cross-validation, AIC); β is optimized directly with no marginal likelihood
- ▶ **LMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the conditional mode of $\mathbf{u}$ is then computed conditionally on those estimates

The distinction does not quite hold for GLMMs

- ▶ **Ridge**: λ is fixed externally (cross-validation, AIC); β is optimized directly with no marginal likelihood
- ▶ **LMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the conditional mode of $\mathbf{u}$ is then computed conditionally on those estimates
- ▶ **Non-Gaussian GLMMs**: the posterior $p(\mathbf{u}|\mathbf{y})$ is not Gaussian, so posterior mode $\neq$ posterior mean; the conditional mode requires integration, not just optimization

The distinction does not quite hold for GLMMs

- ▶ **Ridge**: λ is fixed externally (cross-validation, AIC); β is optimized directly with no marginal likelihood
- ▶ **LMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the conditional mode of $\mathbf{u}$ is then computed conditionally on those estimates
- ▶ **Non-Gaussian GLMMs**: the posterior $p(\mathbf{u}|\mathbf{y})$ is not Gaussian, so posterior mode $\neq$ posterior mean; the conditional mode requires integration, not just optimization
- ▶ The equivalence holds **exactly** only for Gaussian models with known variance components

GAM formulation

GAMs extend this idea: instead of penalizing the size of β , we penalize the **wiggleness** of a smooth function.

A Generalized Additive Model replaces linear terms with smooth functions:

$$g\{E(y_i)\} = \alpha + f_1(x_{i1}; \beta_1) + f_2(x_{i2}; \beta_2) + \dots \quad (5)$$

GAM formulation

GAMs extend this idea: instead of penalizing the size of β , we penalize the **wiggleness** of a smooth function.

A Generalized Additive Model replaces linear terms with smooth functions:

$$g\{E(y_i)\} = \alpha + f_1(x_{i1}; \beta_1) + f_2(x_{i2}; \beta_2) + \dots \quad (5)$$

- ▶ $f_j(\cdot)$ are smooth, flexible functions estimated from data
- ▶ Linear terms $x\beta$ are a special case
- ▶ Random effects can be included

GAM objective function

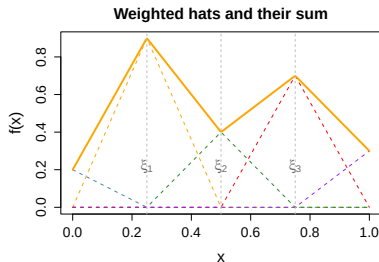
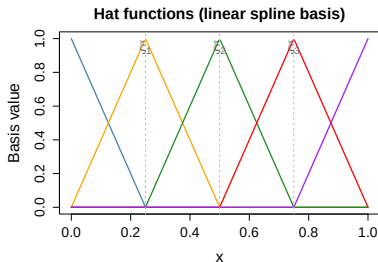
Estimated by maximizing the penalized log-likelihood:

$$\sum_{i=1}^n \log f(y_i; \mu_i) - \lambda \int \{f''(x)\}^2 dx = \sum_{i=1}^n \log f(y_i; \mu_i) - \lambda \boldsymbol{\beta}^\top \mathbf{S} \boldsymbol{\beta} \quad (6)$$

- ▶ First term: fit to data (any exponential family response)
- ▶ $\int \{f''(x)\}^2 dx$: integral over the domain of f , not the data — measures total curvature; evaluates to $\boldsymbol{\beta}^\top \mathbf{S} \boldsymbol{\beta}$ in matrix form
- ▶ λ balances fit against smoothness

What is a spline?

- ▶ A spline is a **weighted sum of basis functions**, each non-zero only near its knot
- ▶ Knots $\xi_1, \dots, \xi_K$ divide the range of x into segments
- ▶ **Hat functions** (tent-shaped) form the basis of a linear spline: continuous, but with kinks at knots
- ▶ Higher-degree bases (e.g., cubic) give smoother splines with continuous derivatives at knots



Basis functions: building blocks

Any smooth $f(x)$ can be represented as a **weighted sum of basis functions** $b_j(x)$:

$$f(x; \boldsymbol{\beta}) = \sum_{j=1}^K \beta_j b_j(x) = \mathbf{b}(x)^\top \boldsymbol{\beta} \quad (7)$$

Basis functions: building blocks

Any smooth $f(x)$ can be represented as a **weighted sum of basis functions** $b_j(x)$:

$$f(x; \beta) = \sum_{j=1}^K \beta_j b_j(x) = \mathbf{b}(x)^\top \beta \quad (7)$$

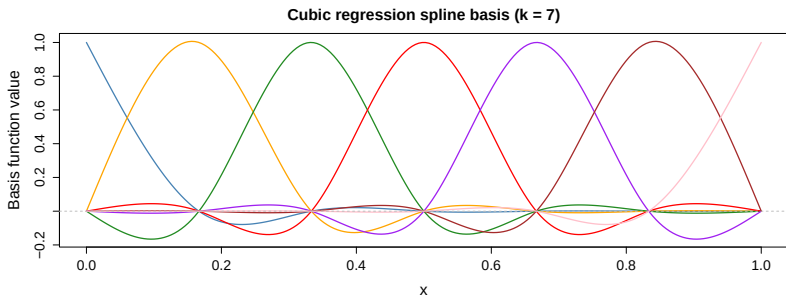
This is the same $\mathbf{X}\beta$ structure as a GLM; we simply **redefine the design matrix $\mathbf{X}$** to contain basis function columns:

$$\mathbf{X}\beta, \quad \text{where } X_{ij} = b_j(x_i) \quad (8)$$

and add penalisation.

Cubic regression splines

The cubic regression spline (CRS) basis uses cubic polynomials between knots, constrained to have continuous first and second derivatives at knots.



Each basis function peaks near its knot but is non-zero across the whole domain.

Thin plate regression splines

The **default in `mgcv`** (Wood 2003)

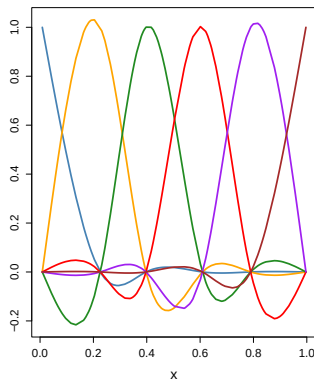
- ▶ No knots to choose; optimises over all smooth functions
- ▶ Uses a low-rank approximation for large n : keeps the k most important basis functions
- ▶ Generally the safest default choice

Spline summary

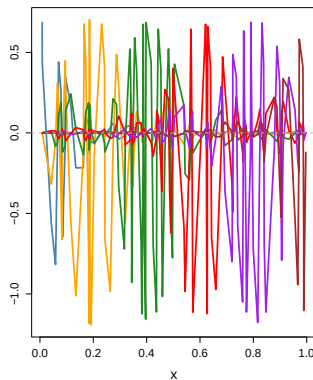
Type	mgcv code	When to use
Thin plate (default)	<code>s(x)</code>	General purpose
Cubic regression	<code>s(x, bs="cr")</code>	Interpretable, fast
P-spline	<code>s(x, bs="ps")</code>	Large datasets
Cyclic cubic	<code>s(x, bs="cc")</code>	Periodic predictors
Duchon spline	<code>s(x, bs="ds")</code>	Spatial data

Building a smooth: step by step

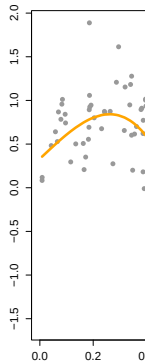
1. Basis functions



2. Weighted by coefficients



3. S



Wiggleness penalty

Without a penalty, enough basis functions can fit *any* dataset perfectly (overfitting).

Wiggleness penalty

Without a penalty, enough basis functions can fit *any* dataset perfectly (overfitting).

The **wiggleness penalty** discourages overly complex smooths:

$$\text{PRSS} = \underbrace{\sum_{i=1}^n \{y_i - f(x_i)\}^2}_{\text{Fit to data}} + \lambda \underbrace{\int \{f''(x)\}^2 dx}_{\text{Wiggleness}} \quad (9)$$

Wiggleness penalty

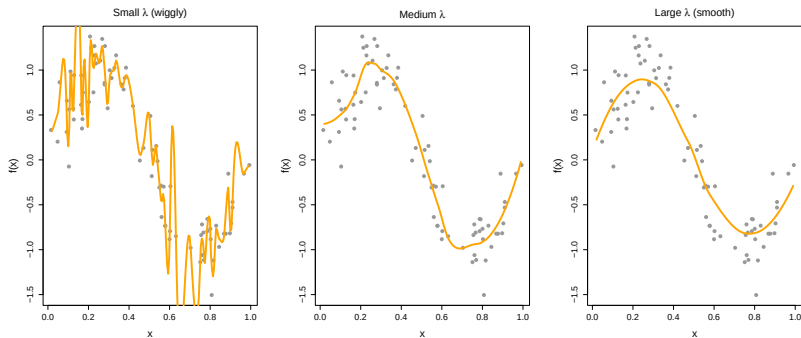
Without a penalty, enough basis functions can fit *any* dataset perfectly (overfitting).

The **wiggleness penalty** discourages overly complex smooths:

$$\text{PRSS} = \underbrace{\sum_{i=1}^n \{y_i - f(x_i)\}^2}_{\text{Fit to data}} + \lambda \underbrace{\int \{f''(x)\}^2 dx}_{\text{Wiggleness}} \quad (9)$$

- ▶ $f''(x)$ is the second derivative (measures curvature)
- ▶ $\lambda \geq 0$ is the **smoothing parameter**
- ▶ $\lambda = 0$: interpolating spline (overfits)
- ▶ $\lambda \rightarrow \infty$: straight line (linear regression)

Smoothing parameter λ



The same data; only λ differs.

The penalty in matrix form

For a penalized spline with basis matrix $\mathbf{B}$ and coefficients β :

$$\text{PRSS} = \|\mathbf{y} - \mathbf{B}\beta\|^2 + \lambda \beta^\top \mathbf{S} \beta \quad (10)$$

- ▶ $\mathbf{S}$ is the **penalty matrix** encoding the wiggleness constraint
- ▶ For a cubic spline: $S_{jk} = \int b_j''(x) b_k''(x) dx$

The penalty in matrix form

For a penalized spline with basis matrix $\mathbf{B}$ and coefficients β :

$$\text{PRSS} = \|\mathbf{y} - \mathbf{B}\beta\|^2 + \lambda \beta^\top \mathbf{S} \beta \quad (10)$$

- ▶ $\mathbf{S}$ is the **penalty matrix** encoding the wiggleness constraint
- ▶ For a cubic spline: $S_{jk} = \int b_j''(x) b_k''(x) dx$

The solution is:

$$\hat{\beta} = (\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{S})^{-1} \mathbf{B}^\top \mathbf{y} \quad (11)$$

The penalty in matrix form

For a penalized spline with basis matrix $\mathbf{B}$ and coefficients β :

$$\text{PRSS} = \|\mathbf{y} - \mathbf{B}\beta\|^2 + \lambda \beta^\top \mathbf{S} \beta \quad (10)$$

- ▶ $\mathbf{S}$ is the **penalty matrix** encoding the wiggleness constraint
- ▶ For a cubic spline: $S_{jk} = \int b_j''(x) b_k''(x) dx$

The solution is:

$$\hat{\beta} = (\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{S})^{-1} \mathbf{B}^\top \mathbf{y} \quad (11)$$

Compare to ridge: $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$; same structure with $\mathbf{S}$ replacing $\mathbf{I}$

Effective degrees of freedom

The effective degrees of freedom (EDF) of a smooth quantifies its complexity:

$$\text{EDF} = \text{tr}\{(\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{S})^{-1} \mathbf{B}^\top \mathbf{B}\} \quad (12)$$

- | | |
|--|--|
| ▶ EDF = 1: straight line | ▶ Reported by <code>summary.gam</code> |
| ▶ EDF = 2: quadratic | ▶ Used for hypothesis tests |
| ▶ EDF $\approx k$: nearly interpolating | ▶ Larger EDF $\Rightarrow$ more complex smooth |

How do we choose λ ?

Several criteria exist for selecting the smoothing parameter automatically:

1. **GCV**: Generalized Cross-Validation
2. **ML**: Maximum Likelihood
3. **REML**: Restricted Maximum Likelihood

Others exist (NCV, QNCV, P-REML, P-ML — see `?mgcv::gam`).
All treat λ as something to be estimated from the data.

GCV: Generalized Cross-Validation

Proposed by Craven and Wahba (1979), popularized for GAMs by **Wood (2004)**:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{\{n - \text{EDF}(\lambda)\}^2} \quad (13)$$

GCV: Generalized Cross-Validation

Proposed by Craven and Wahba (1979), popularized for GAMs by **Wood (2004)**:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{\{n - \text{EDF}(\lambda)\}^2} \quad (13)$$

- ▶ Approximates leave-one-out cross-validation at $O(n)$ cost
- ▶ Penalizes model complexity via EDF
- ▶ Minimizing GCV over λ balances fit vs. smoothness

GCV: Generalized Cross-Validation

Proposed by Craven and Wahba (1979), popularized for GAMs by **Wood (2004)**:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{\{n - \text{EDF}(\lambda)\}^2} \quad (13)$$

- ▶ Approximates leave-one-out cross-validation at $O(n)$ cost
- ▶ Penalizes model complexity via EDF
- ▶ Minimizing GCV over λ balances fit vs. smoothness

Problem: GCV can undersmooth (too wiggly) in some cases, especially with correlated errors (Reiss and Ogden 2009)

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML (Maximum Likelihood): maximize the marginal likelihood of $\mathbf{y}$ over both β and λ

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML (Maximum Likelihood): maximize the marginal likelihood of $\mathbf{y}$ over both β and λ

REML (Restricted Maximum Likelihood): maximize the likelihood after projecting out fixed effects

- ▶ REML is less prone to undersmoothing than ML (Wood 2011)
- ▶ REML accounts for uncertainty in fixed effects when estimating λ
- ▶ Generally recommended; but **not** the default (GCV.Cp is)

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML (Maximum Likelihood): maximize the marginal likelihood of $\mathbf{y}$ over both β and λ

REML (Restricted Maximum Likelihood): maximize the likelihood after projecting out fixed effects

- ▶ REML is less prone to undersmoothing than ML (Wood 2011)
- ▶ REML accounts for uncertainty in fixed effects when estimating λ
- ▶ Generally recommended; but **not** the default (GCV.Cp is)

Use `method="REML"` in `mgcv::gam()` — specify it explicitly

Comparing criteria

Criterion	Basis	Tends to	Use when
GCV	CV approximation	Undersmooth	Legacy, simple models
ML	Likelihood	Undersmooth	Rarely preferred
REML	Restricted likelihood	Best calibrated	Recommended

Comparing criteria

Criterion	Basis	Tends to	Use when
GCV	CV approximation	Undersmooth	Legacy, simple models
ML	Likelihood	Undersmooth	Rarely preferred
REML	Restricted likelihood	Best calibrated	Recommended

In R:

```
gam(y ~ s(x), method = "REML") # recommended
gam(y ~ s(x), method = "GCV.Cp") # GCV
gam(y ~ s(x), method = "ML") # ML
```

The GLMM representation

(Wood 2004, 2011)

The penalty matrix **S** is rank-deficient: some eigenvalues are zero (directions that are not penalized).

The GLMM representation

(Wood 2004, 2011)

The penalty matrix $\mathbf{S}$ is rank-deficient: some eigenvalues are zero (directions that are not penalized).

Decompose $\mathbf{S}$ via eigen decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and reparameterise the basis matrix into eigenvectors with zero eigenvalues ($\mathbf{B}_0$, unpenalized) and non-zero eigenvalues ($\mathbf{B}_1$, penalized):

$$f(x) = \underbrace{\mathbf{B}_0\beta_0}_{\text{Unpenalized (fixed)}} + \underbrace{\mathbf{B}_1\mathbf{b}}_{\text{Penalized (random)}} \quad (14)$$

The GLMM representation

(Wood 2004, 2011)

The penalty matrix $\mathbf{S}$ is rank-deficient: some eigenvalues are zero (directions that are not penalized).

Decompose $\mathbf{S}$ via eigen decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and reparameterise the basis matrix into eigenvectors with zero eigenvalues ($\mathbf{B}_0$, unpenalized) and non-zero eigenvalues ($\mathbf{B}_1$, penalized):

$$f(x) = \underbrace{\mathbf{B}_0\boldsymbol{\beta}_0}_{\text{Unpenalized (fixed)}} + \underbrace{\mathbf{B}_1\mathbf{b}}_{\text{Penalized (random)}} \quad (14)$$

with $\mathbf{b} \sim \mathcal{N}(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I})$

The GLMM representation

(Wood 2004, 2011)

The penalty matrix $\mathbf{S}$ is rank-deficient: some eigenvalues are zero (directions that are not penalized).

Decompose $\mathbf{S}$ via eigen decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and reparameterise the basis matrix into eigenvectors with zero eigenvalues ($\mathbf{B}_0$, unpenalized) and non-zero eigenvalues ($\mathbf{B}_1$, penalized):

$$f(x) = \underbrace{\mathbf{B}_0\boldsymbol{\beta}_0}_{\text{Unpenalized (fixed)}} + \underbrace{\mathbf{B}_1\mathbf{b}}_{\text{Penalized (random)}} \quad (14)$$

with $\mathbf{b} \sim \mathcal{N}(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I})$

This is **exactly** a GLMM: fixed-effects design matrix $\mathbf{X} = \mathbf{B}_0$ and

What this means in practice

- ▶ GAMs can be fitted using GLMM software
- ▶ The smoothing parameter λ is determined by the random-effect variance: $\sigma_b^2 = \sigma^2 / \lambda$
- ▶ REML estimation of λ is exactly REML estimation of the random-effect variance
- ▶ A GAM smooth can be represented as a GLMM random effect

What this means in practice

- ▶ GAMs can be fitted using GLMM software
- ▶ The smoothing parameter λ is determined by the random-effect variance: $\sigma_b^2 = \sigma^2 / \lambda$
- ▶ REML estimation of λ is exactly REML estimation of the random-effect variance
- ▶ A GAM smooth can be represented as a GLMM random effect

A GAM smooth can be expressed as a GLMM random effect; useful for estimation, not a full equivalence

GAMs in glmmTMB

Since the GAM smooth is a GLMM, it can be fitted in glmmTMB using `mgcv` smooth terms:

```
library(glmmTMB)
model <- glmmTMB(y ~ s(x), family = poisson, data = dat)
```

GAMs in glmmTMB

Since the GAM smooth is a GLMM, it can be fitted in glmmTMB using mgcv smooth terms:

```
library(glmmTMB)
model <- glmmTMB(y ~ s(x), family = poisson, data = dat)
```

You can combine GAM smooths with random effects in the same model:

```
model <- glmmTMB(y ~ s(x) + (1|group),
                 family = poisson, data = dat)
```

GAMs in glmmTMB

Since the GAM smooth is a GLMM, it can be fitted in `glmmTMB` using `mgcv` smooth terms:

```
library(glmmTMB)
model <- glmmTMB(y ~ s(x), family = poisson, data = dat)
```

You can combine GAM smooths with random effects in the same model:

```
model <- glmmTMB(y ~ s(x) + (1|group),
                 family = poisson, data = dat)
```

- ▶ Uses the Wood (2004, 2011) diagonalization internally
- ▶ Enables non-Gaussian responses with GAM-type smoothing

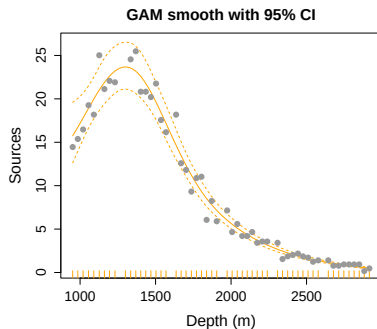
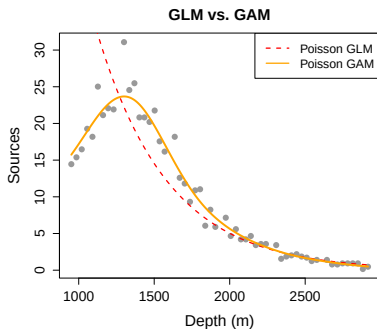
Our example data: bioluminescence at depth

Data from Gillibrand et al. (2007), via Zuur et al. (2009)

- ▶ 1,654 observations across 19 stations
- ▶ Response: number of bioluminescent sources
- ▶ Predictor: sample depth (m)
- ▶ Strong non-linear relationship with depth

Station	SampleDepth	Sources
1	517	28.73
1	582	27.90
1	547	23.44
1	614	18.33
1	1068	12.38

ISIT: GLM vs. GAM



ISIT: fitting

```
library(mgcv)
ISIT <- read.csv("../data/ISIT.csv")
dat_ex <- subset(ISIT, Station == 9)

model_gam <- gam(Sources ~ s(SampleDepth, bs="cr"),
                  family = Gamma(link="log"), data = dat_ex,
                  method = "REML")
```

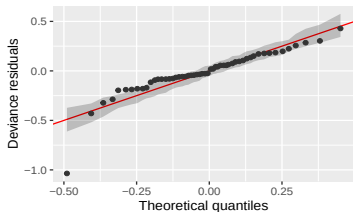
ISIT: summary

```
##
## Family: Gamma
## Link function: log
##
## Formula:
## Sources ~ s(SampleDepth, bs = "cr")
##
## Parametric coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.70174    0.02749    61.9   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F p-value
## s(SampleDepth) 5.358  6.472 310.6 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

ISIT: model checking

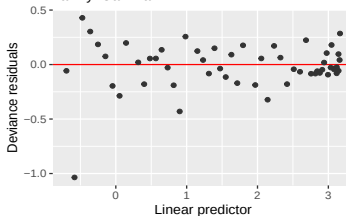
QQ plot of residuals

Method: simulate

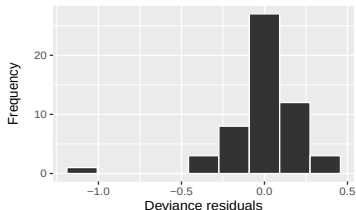


Residuals vs linear predictor

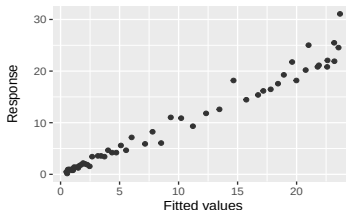
Family: Gamma



Histogram of residuals



Observed vs fitted values



ISIT: multiple stations

```
model_full <- gam(Sources ~ s(SampleDepth, bs="cr") +
                  s(Station, bs="re"),
                  family = Gamma(link="log"), data = ISIT,
                  method = "REML")
```

- ▶ `s(Station, bs="re")`: random intercept for each station (a GAMM)
- ▶ The same in `glmmTMB`:

```
library(glmmTMB)
model_tmb <- glmmTMB(Sources ~ s(SampleDepth, bs="cr") +
                     (1|Station),
                     family = Gamma(link="log"), data = ISIT)
```

Our example data: road-killed toads

Data from Zuur et al. (2009), originally Mira et al.

- ▶ 52 road sectors in Portugal
- ▶ Response: number of road-killed *Bufo calamita* (natterjack toad)
- ▶ Predictor: proportion of open land in sector (OPEN.L)
- ▶ Non-linear relationship expected; road kills vary non-linearly with open habitat

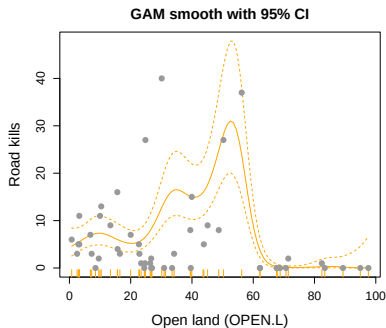
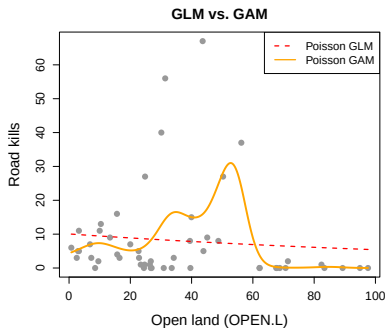
BufoCalamita	OPEN.L
5	22.684
1	24.657
40	30.121
27	50.277
67	43.609

Road kills: fitting

```
rk <- read.csv("../data/RoadKills.csv")

model_rk <- gam(BufoCalamita ~ s(OPEN.L, bs="cr"),
                family = poisson, data = rk)
```

Road kills: GLM vs. GAM



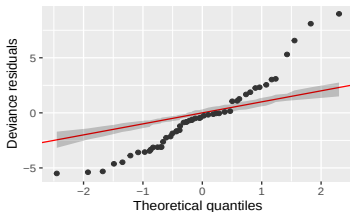
Road kills: summary

```
##
## Family: poisson
## Link function: log
##
## Formula:
## BufoCalamita ~ s(OPEN.L, bs = "cr")
##
## Parametric coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.2645     0.1642   7.703 1.33e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##             edf Ref.df Chi.sq p-value
## s(OPEN.L)  8.853  8.992  157.9  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Road kills: model checking

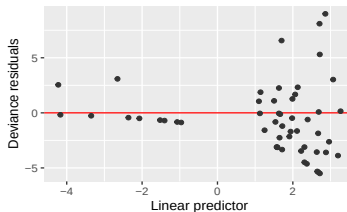
QQ plot of residuals

Method: simulate

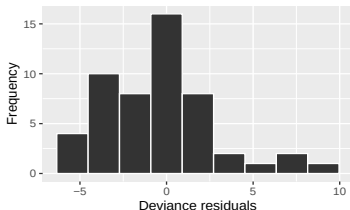


Residuals vs linear predictor

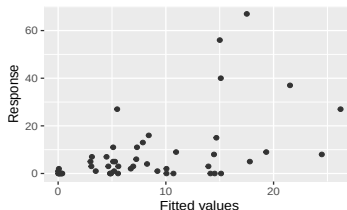
Family: poisson



Histogram of residuals

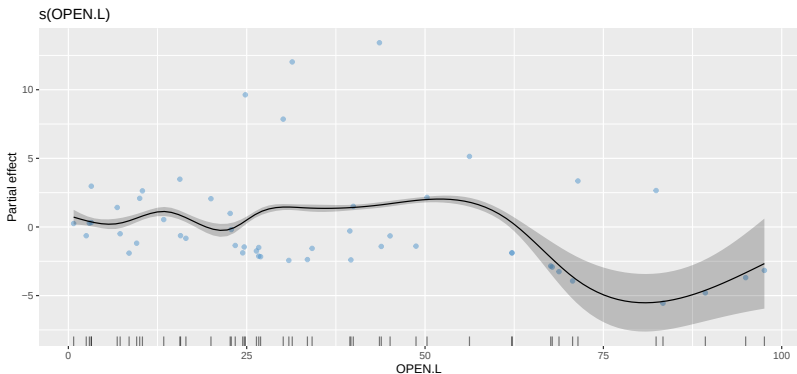


Observed vs fitted values



Road kills: visualisation with gratia

```
library(gratia)
draw(model_rk, residuals=TRUE)
```



Road kills: adding more predictors

```

model_rk2 <- gam(BufoCalamita ~ s(OPEN.L, bs="cr") +
                  s(D.WAT.COUR, bs="cr") +
                  s(L.D.ROAD, bs="cr"),
                  family = poisson, data = rk, method = "REML")
  
```

- ▶ Multiple smooths can be combined additively
- ▶ Each smooth's edf is estimated and penalized independently
- ▶ Use AIC or REML score to compare models

Summary

- ▶ **Ridge regression:** penalizes $\|\beta\|^2$, equivalent to a normal prior; conditional mode = MAP estimate
- ▶ **GAMs** extend GLMs with smooth functions $f(x)$ instead of linear terms $x\beta$
- ▶ **Splines** represent $f(x)$ as weighted basis functions; many types available
- ▶ **Wigginess penalty** controls smoothness; λ is estimated automatically
- ▶ **REML** is the recommended criterion for smoothing parameter selection
- ▶ **GAMs and GLMMs are connected:** a penalized smooth can be expressed as a random effect

End

